An Adaptive Stepsize RRT Planning Algorithm for Open-Chain Robots

Byungchul An, *Student Member, IEEE*, Jinkyu Kim, *Student Member, IEEE*, and Frank C. Park, *Fellow, IEEE*

***Abstract*—Motion planning algorithms that rely upon the randomly exploring random tree (RRT) typically require the user to choose an appropriate stepsize; this is generally a highly problem-dependent and time-consuming process requiring trial and error. We propose an adaptive stepsize RRT path planning algorithm for open-chain robots in which only a minimum obstacle size parameter is required as input. Exploiting the structure of an open chain's forward kinematics as well as a standard inequality bound on the operator-induced matrix norm, we derive a maximum Cartesian displacement bound between two configurations of the same robot, and use this bound to determine a maximum allowable stepsize at each iteration. Numerical experiments involving a ten-DOF planar open chain and a seven-axis industrial robot arm demonstrate the practical advantages of our algorithm over standard fixed-stepsize RRT planning algorithms.**

***Index Terms*—Adaptive stepsize, operator norm, path planning, rapidly-exploring random tree.**

I. Introduction

PLANNING algorithms that rely upon the randomly-exploring random tree (RRT) [1] are some of the most widely used probabilistic sampling-based methods for robot path planning today. The basic RRT algorithm is efficient and robust, easy to implement, and generalizes readily to more complex robots (e.g., closed chains) and to diverse constraints (e.g., velocity and actuator limits, dynamic balance requirements). Many extensions and improvements to the basic RRT algorithm have been presented in the literature; see, e.g., [2]–[4] and the references cited therein.

A typical RRT-based algorithm requires the user to specify a stepsize parameter that sets the distance between the current node and a newly generated node (Fig. 1): a larger stepsize implies wider spacing between connected nodes, so that a feasible path can generally be found with a sparser tree (and thus more quickly). If the stepsize is set too large, however, the resulting path may not be feasible: because collision checking is time-consuming and usually performed only at the nodes, possible collisions with small objects (i.e., those that are small enough to fit between two connected nodes) may go undetected. The effects of the choice of stepsize on RRT performance are further investigated in [4]. If the size of the smallest workspace object is known in advance, then choosing the largest possible stepsize that still prevents these smallest objects from fitting between two connected nodes is one practical means to ensure collision-free paths. Choosing an appropriate stepsize is often a time-consuming trial-and-error process that makes RRT, despite its simplicity and well-documented advantages, challenging to use in practice.

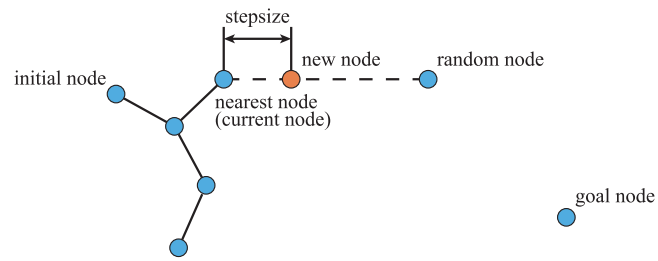


Fig. 1. The stepsize and the named nodes for RRT.

Several variable stepsize RRT methods have been reported in the literature. RRT-Connect [5], which takes multiple steps of a fixed stepsize toward the goal node until a collision occurs or the goal is reached, develops a number of heuristics that improve the algorithm's overall speed vis-à-vis basic RRT. [6] observes that tree expansion tends to become more difficult when in close proximity to collisions, and proposes additional heuristics to RRT-Connect that retains only 90% of the valid part of an extended tree. Variable stepsize RRT planning algorithms have also been proposed for the path planning of UAVs, wheeled vehicles, and other robots that can be modeled as a single rigid body. In [7] a randomly sampled node is connected to its nearest node if there is no obstacle between these nodes. If a collision occurs, the random node is not added to the tree, but stored as a collision point; the set of collision points are then used later to find the shortest path. In [8], the stepsize is increased in proportion to the number of parent nodes that lie between the current node and the root (initial or goal) node, with the aim of increasing the area of exploration over a fixed period.

In this letter we propose an adaptive stepsize RRT planning algorithm for open kinematic chains. Like most RRT planners, ours constructs a tree in the joint configuration space, while

Manuscript received April 15, 2017; accepted July 31, 2017. Date of publication August 29, 2017; date of current version September 8, 2017. This letter was recommended for publication by Associate Editor F. van der Stappen and Editor N. Amato upon evaluation of the reviewers' comments. This work was supported in part by the SNU BMRR Center (DAPA-UD130070ID), SNU-IAMD, BK21+, MI Technology Innovation Program (10048320), and Soft Robotics Research Center. *(Corresponding author: Frank C. Park.)*

B. An is with the Robotics Laboratory, Seoul National University, Seoul 08826, South Korea, and also with the Healthcare Robotics Research Group, Korea Institute of Technology, Seoul 02792, South Korea (e-mail: abc8512@snu.ac.kr).

J. Kim and F. C. Park are with the Robotics Laboratory, Seoul National University, Seoul 08826, South Korea (e-mail: zkyu@snu.ac.kr; fcp@snu.ac.kr).

Digital Object Identifier 10.1109/LRA.2017.2745542

2377-3766 © 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on December 15,2025 at 09:28:35 UTC from IEEE Xplore. Restrictions apply.

collision checking is done in the Cartesian space occupied by the robot links. Using a standard inequality on the matrix norm induced from a vector norm, we derive an approximate bound on the Cartesian displacement of the open chain tip for a given joint space displacement. This bound, which can be computed in real-time and is useful for a range of motion planning contexts, is then used to compute, for a given robot configuration, approximate bounds on the maximal deviations of the points on each of the robot's links (each link is modeled as a convex polytope). In this regard the product of exponentials forward kinematics formula [9] plays an essential role in the derivation and efficient computation of these bounds. Not having to manually set the stepsize parameter through a time-consuming trial-and-error process is an important feature of our algorithm. The stepsize bounds can also be used to improve the performance of other existing RRT algorithms that adaptively vary the stepsize or take multiple steps of a given stepsize.

The letter is organized as follows. The main inequality bound is derived in Section II, while the accompanying adaptive stepsize RRT algorithm is presented in Section III. Section IV presents experimental results using our planning algorithm for a ten-dof planar open chain and a seven-axis industrial robot manipulator. We conclude with some suggestions for extension to more complex chains, and also to planning in spaces of reduced dimension.

II. A Cartesian Displacement Bound for Open Chains

We first cover some preliminaries on matrix norms; see, e.g., [11] for a review of standard results on matrix and operator norms. Given two vector norms $\|\cdot\|_X$ on $\mathbb{R}^n$, $\|\cdot\|_Y$ on $\mathbb{R}^m$, and a matrix $A \in \mathbb{R}^{m\times n}$, the matrix operator norm induced from the two vector norms is defined as follows:

$$\|A\|_{XY} = \sup_{x\neq 0} \frac{\|Ax\|_Y}{\|x\|_X}. \tag{1}$$

The following inequality then always holds:

$$\|Ax\|_Y \leq \|A\|_{XY} \cdot \|x\|_X$$

For the basic vector norms, e.g., the 1-norm $\|\cdot\|_1$, the Euclidean norm $\|\cdot\|_2$, and the sup-norm $\|\cdot\|_\infty$, the corresponding induced matrix norm applied to $A = (a_{ij}) \in \mathbb{R}^{m\times n}$ are respectively

$$\|A\|_{1,1} = \max_j \sum_{i=1}^m |a_{ij}|$$

$$\|A\|_{2,2} = \rho_{\max}(A) = \sqrt{\lambda_{\max}(A^T A)}$$

$$\|A\|_{\infty,\infty} = \max_i \sum_{j=1}^n |a_{ij}|.$$

It is well-known that the Frobenius norm $\|\cdot\|_F$, defined as

$$\|A\|_F = \sqrt{\mathrm{Tr}(A^T A)}$$

is not an induced norm of any vector norm. The following inequality also holds for all $A \in \mathbb{R}^{n\times n}$:

$$\|A\|_2 \leq \|A\|_F \leq \sqrt{n}\|A\|_2.$$

We now derive our main result which provides, for an open kinematic chain whose task space is the special Euclidean group SE(3) of rigid body transformations, an inequality bound on the maximum Cartesian position displacement in the task space for a given displacement in the input configuration space.

Consider an m-degree of freedom open chain whose forward kinematics is represented in the following product of exponentials form [9], [12], [13]:

$$f(\theta) = e^{[\mathcal{S}_1]\theta_1} \cdots e^{[\mathcal{S}_m]\theta_m} M, \tag{2}$$

where θ_i, $i = 1, \ldots, m$ denote the joint variables, and the 4×4 matrices $M \in \mathrm{SE}(3)$ and $[\mathcal{S}_i] \in \mathrm{se}(3)$ (here se(3) denotes the Lie algebra of SE(3)) are of the form

$$M = \begin{bmatrix} R_M & p_M \\ 0 & 1 \end{bmatrix}, \quad [\mathcal{S}_i] = \begin{bmatrix} [\omega_i] & v_i \\ 0 & 0 \end{bmatrix}, \tag{3}$$

with $R_M \in \mathrm{SO}(3)$, $p_M \in \mathbb{R}^3$, (for convenience we will also use the more compact notation $M = (R_M, p_M) \in \mathrm{SE}(3)$), $v_i \in \mathbb{R}^3$, and for a three-dimensional vector $\omega \in \mathbb{R}^3$, the square bracket notation $[\omega]$ is defined as the following 3×3 skew-symmetric matrix representation of ω:

$$[\omega] = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}. \tag{4}$$

$[\mathcal{S}_i]$ will be written in the equivalent six-dimensional column vector form as $\mathcal{S}_i \in \mathbb{R}^6$, and also using the notation $\mathcal{S}_i = (\omega_i, v_i) \in \mathbb{R}^6$. For any $X = (R, p) \in SE(3)$, the 6×6 adjoint matrix $[\mathrm{Ad}_X]$ is defined as follows:

$$[\mathrm{Ad}_X] = \begin{bmatrix} R & 0 \\ [p]\,R & R \end{bmatrix}. \tag{5}$$

Letting $p \in \mathbb{R}^3$ be the Cartesian position of the end-effector frame expressed in fixed frame coordinates, its velocity $\dot{p}$ is then given by

$$\dot{p} = \omega_s \times p + v_s, \tag{6}$$

where

$$\begin{bmatrix} \omega_s \\ v_s \end{bmatrix} = J_s(\theta)\dot{\theta} = [\mathcal{S}_1' \ \cdots \ \mathcal{S}_m']\,\dot{\theta} \tag{7}$$

$$\mathcal{S}_i' = \left[\mathrm{Ad}_{e^{[\mathcal{S}_1]\theta_1}\cdots e^{[\mathcal{S}_{i-1}]\theta_{i-1}}}\right]\mathcal{S}_i, \tag{8}$$

with the adjoint matrix $[\mathrm{Ad}_{(\cdot)}]$ as defined in (5). Each of the six-dimensional vectors $\mathcal{S}_i' = (\omega_i', v_i')$ depends explicitly on θ, and represents the screw motion corresponding to the i-th joint axis when the robot is at the configuration θ.

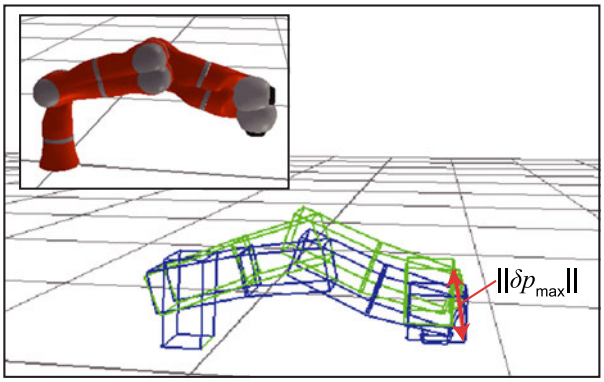
With the above preliminaries we now state our main result.

Proposition: Given an m-joint open chain robot with forward kinematics (2), for a joint displacement $\theta \mapsto \theta + \delta\theta$, the end-effector Cartesian position displacement $p \mapsto p + \delta p$ is approximately bounded by $\|\delta p\|_Y \leq \|A\|_{XY} \cdot \|\delta\theta\|_X$, where

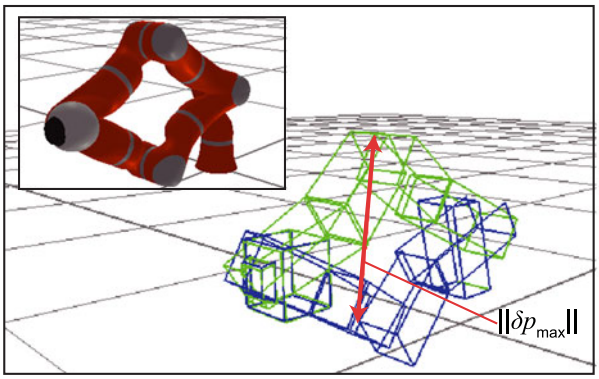
$$A = [\omega_1' \times p + v_1' | \cdots | \omega_m' \times p + v_m'] \in \mathbb{R}^{3\times m}, \tag{9}$$

$\|\cdot\|_X$ denotes a norm in joint configuration space, $\|\cdot\|_Y$ denotes a norm in the Cartesian position space $\mathbb{R}^3$, and $\|\cdot\|_{XY}$ denotes the induced matrix operator norm as defined in (1).

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on December 15,2025 at 09:28:35 UTC from IEEE Xplore. Restrictions apply.



(a)



(b)

Fig. 2. Two examples of the maximum Cartesian space displacement $\|\delta p_{\max}\|$ of the same open chain having different pairs of initial-final configurations: (a) $\|\delta p_{\max}\|$ occurs at the end-effector; (b) $\|\delta p_{\max}\|$ occurs at the elbow. Each link is modeled as a three-dimensional rectangular box.

Proof: Noting that $\dot{p} = \omega_s \times p + v_s$ with ω_s and v_s obtained from (7)–(8), $\dot{p}$ can be written explicitly as

$$\dot{p} = \sum_{i=1}^{m} ([\omega_i']p + v_i')\,\dot{\theta}_i \tag{10}$$

$$= [[\omega_1']p + v_1' \mid \cdots \mid [\omega_m']p + v_m']\,\dot{\theta}, \tag{11}$$

from which it follows that

$$\delta p \approx [\omega_1' \times p + v_1' \mid \cdots \mid \omega_m' \times p + v_m']\ \delta\theta. \tag{12}$$

The approximate bound on $\|\delta p\|$ now follows. ■

Note that the vector $\omega_i' \times p + v_i' \in \mathbb{R}^3$ represents the contribution of the i-th joint velocity $\dot{\theta}_i$ to the overall end-effector velocity $\dot{p}$.

Now suppose each link i has the shape of a convex polytope, and let $\mathcal{B}_i = \{\mathbf{p}_{i1}, \mathbf{p}_{i2}, \ldots\}$ be the set of link vertices for link i. Let $p_{ij} \in \mathbb{R}^3$ be the coordinates of link vertex $\mathbf{p}_{ij}$ expressed in fixed frame coordinates. Given some joint space displacement $\delta\theta \in \mathbb{R}^m$ from its current configuration θ, there will exist some link vertex $\mathbf{p}_{\max}$ on the robot at which the corresponding Cartesian space displacement will be maximized; denote this maximum displacement by $\delta p_{\max} \in \mathbb{R}^3$ (Fig. 2 illustrates the maximum Cartesian space displacement for two different pairs of initial-final configurations). The following corollary then holds.

Corollary 1: The maximum Cartesian space displacement of the m-link open chain is approximately bounded by

$$\begin{aligned}\|\delta p_{\max}\|_Y &= \max_{i,j} \|\delta p_{ij}\|_Y \\ &\leq \max_{i,j} \|A_{ij}\|_{XY} \|\delta\theta\|_X\end{aligned} \tag{13}$$

where

$$A_{ij} = [\omega_1' \times p_{ij} + v_1' \mid \cdots \mid \omega_i' \times p_{ij} + v_i'], \tag{14}$$

and the search for the maximum occurs over the range $i = 1, \ldots, m$, $p_{ij} \in \mathcal{B}_i$.

The proof follows straightforwardly from a repeated application of the earlier proposition.

A similar result can be derived for input spaces other than the joint space, e.g., a lower-dimensional space, called the latent space, together with a differentiable mapping from the latent space to the joint space. Such situations arise when, e.g., dimension reduction methods are employed to construct a lower-dimensional representation of the configuration space based on a collection of sample trajectories (see, for example, [10]). Let $g : \mathbb{R}^d \to \mathbb{R}^m$, $x \mapsto \theta = g(x)$ be the local coordinate representation of the differentiable mapping from the latent space to the joint configuration space, with $d \leq m$. Let $\delta x \in \mathbb{R}^d$ be a displacement in latent space and $\delta p_{\max}$ be the corresponding maximum Cartesian space displacement for a point on the robot. Denote by $\|\cdot\|_Z$ the norm on $\mathbb{R}^d$. The following corollary then holds.

Corollary 2: The maximum Cartesian space displacement $\delta p_{\max}$ is approximately bounded by

$$\|\delta p_{\max}\|_Y \leq \max_{i,j} \|A_{ij}\|_{XY} \left\| \frac{\partial g}{\partial x} \right\|_{ZX} \|\delta x\|_Z. \tag{15}$$

The corollary follows directly from the fact that $\delta\theta \approx \frac{\partial g}{\partial x}\delta x$.

III. Adaptive Stepsize RRT

Based on the inequalities derived in the previous section, we now describe our adaptive stepsize RRT planning algorithm. A maximum Cartesian displacement value Δ in the workspace is first specified; for example, Δ can be set to be the width of the smallest obstacle in the workspace. Setting the right-hand side of (13) to Δ, i.e.,

$$\|\delta p_{\max}\| \leq \max_{i,j} \|A_{ij}\| \|\delta\theta\| = \Delta, \tag{16}$$

at each joint space configuration the stepsize is then set to

$$\|\delta\theta\| = \frac{\Delta}{\max_{i,j} \|A_{ij}\|}. \tag{17}$$

Similarly, for planning in the latent space, the right-hand side of (15) is set to Δ, and the stepsize is determined as follows:

$$\|\delta p_{\max}\| \leq \max_{i,j} \|A_{ij}\| \left\| \frac{\partial g}{\partial x} \right\| \|\delta x\| = \Delta \tag{18}$$

$$\|\delta x\| = \frac{\Delta}{\max_{i,j} \|A_{ij}\| \left\| \frac{\partial g}{\partial x} \right\|}. \tag{19}$$

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on December 15,2025 at 09:28:35 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Bounded Displacement Adaptive Stepsize RRT.

1: Tree, Δ
2: Tree.AddNode(θ_{init})
3: **While** $i < I_{\max}$ **do**
4: $i \leftarrow i + 1$
5: $\theta_{\text{rand}} \leftarrow$ RandomSampling()
6: $\theta_{\text{near}} \leftarrow$ FindNearestNodeOnTree(Tree, θ_{rand})
7: $\hat{v}_{\text{new}} \leftarrow$ GetNewDirection(θ_{near}, θ_{rand}, Tree)
8: $\|\delta\theta\| \leftarrow$ GetStepsize(θ_{near}, $\hat{v}_{\text{new}}$, Δ)
9: $\theta_{\text{new}} \leftarrow$ Extend(Tree, θ_{near}, $\hat{v}_{\text{new}}$, $\|\delta\theta\|$)
10: **if** CheckConstraints(θ_{new}, constraints) **then**
11: Tree.AddNode(θ_{new})
12: **if** Connect(θ_{new}, θ_{goal}, Δ) **then**
13: [Tree, *flag*] $\leftarrow$ ExtendFurther(Tree, θ_{new}, θ_{goal}, $\hat{v}_{\text{new}}$)
14: **if** *flag* $==$ *true* **then**
15: **return** Path(Tree)
16: **end if**
17: **end if**
18: **end if**
19: **end While**
20: **return** NULL

Algorithm 2: GetStepsize(θ, $\hat{v}$, Δ).

1: $c \leftarrow \max_{i,j} \|A_{ij}(\theta)\|$
2: $\|\delta\theta\| \leftarrow \frac{\Delta}{c}\|\hat{v}\|$
3: **return** $\|\delta\theta\|$

Algorithm 3: Connect(θ_{a}, θ_{b}, Δ).

1: $\mathcal{B}_{\text{a}} \leftarrow$ GetEachVertexPosition(θ_{a})
2: $\mathcal{B}_{\text{b}} \leftarrow$ GetEachVertexPosition(θ_{b})
3: **if** $\max_{i,j} \|p_{ij}(\theta_{\text{a}}) - p_{ij}(\theta_{\text{b}})\| \leq \Delta$ **then**
4: **return** *true*
5: **else**
6: **return** *false*
7: **end if**

Note that the stepsizes in (17) and (19) depend on the robot's current configuration, and tend to be smaller when a robot's links are fully extended, and larger when the links are folded.

Algorithm 1 describes the main steps of our bounded displacement adaptive stepsize RRT algorithm. Lines 1–7 are the same as for the standard joint space RRT algorithm: sample a random node (θ_{rand}) in the configuration space, find its nearest node (θ_{near}) on the tree and the direction ($\hat{v}_{\text{new}}$) in which to extend the tree. The adaptive stepsize (17) is evaluated (line 8) and the new node θ_{new} generated in the direction of $\hat{v}_{\text{new}}$ (line 9). If the new node is collision-free and within the allowed joint range (line 10), then it is added to the tree (line 11).

Lines 12–17 implement the adaptive stepsize planning procedure. *Connect()* (Algorithm 3) determines if the two nodes θ_{new} and θ_{goal} should be connected. In the most basic form of fixed stepsize RRT planning, the connection test is typically done in the joint configuration space, e.g., θ_{new} and θ_{goal} are

Algorithm 4: ExtendFurther(Tree, θ, θ_{goal}, $\hat{v}$).

1: **While** $j < J_{\max}$ **do**
2: $\|\delta\theta\| \leftarrow$ GetStepsize(θ, $\hat{v}$, Δ)
3: $\theta' \leftarrow$ Extend(Tree, θ, $\hat{v}$, $\|\delta\theta\|$)
4: **if** CheckConstraints(θ', constraints) **then**
5: Tree.AddNode(θ')
6: **if** ($||\theta' - \theta_{\text{goal}}|| \leq \|\delta\theta\|$) **then**
7: **return** [Tree, *true*]
8: **else**
9: $\theta \leftarrow \theta'$
10: **end if**
11: **else**
12: **return** [Tree, *false*]
13: **end if**
14: $j \leftarrow j + 1$
15: **end While**
16: **return** [Tree, *false*]

connected if the condition $\|\theta_{\text{new}} - \theta_{\text{goal}}\| \leq \epsilon$ is satisfied for some arbitrary user-prescribed ϵ. In our case it makes more sense to use $\|\delta\theta\|_{\theta=\theta_{\text{new}}}$ in place of ϵ, or noting from (17) that $\Delta = \max_{i,j} \|A_{ij}\| \cdot \|\delta\theta\|$, to connect θ_{new} and θ_{goal} if

$$\max_{i,j} \|A_{ij}\| \cdot \|\theta_{\text{new}} - \theta_{\text{goal}}\| \leq \Delta \tag{20}$$

is satisfied. However, since from Corollary 1 we have

$$\begin{aligned} \|\delta p_{\max}\| &= \max_{i,j} \|p_{ij}(\theta_{\text{new}}) - p_{ij}(\theta_{\text{goal}})\| \\ &\leq \max_{i,j} \|A_{ij}\| \cdot \|\theta_{\text{new}} - \theta_{\text{goal}}\|, \end{aligned}$$

Connect() uses the less restrictive connection test $\|\delta p_{\max}\| \leq \Delta$ to speed up planning. In our later experiments we compare the results of using both the joint space and Cartesian space metrics for the connection test.

If θ_{new} and θ_{goal} are close enough to pass the above connection test, then in principle they could be connected and the algorithm terminated. In practice, θ_{new} and θ_{goal} may be quite distant from each other relative to other connected node pairs (we have observed in our later planar robot experiments that these values sometimes exceed more than thirty times typical values of $\|\theta_i - \theta_{i+1}\|$). Although the condition $\|\delta p_{\max}\| \leq \Delta$ ensures that there is no obstacle larger than Δ between $p_{ij}(\theta_{\text{new}})$ and $p_{ij}(\theta_{\text{goal}}), \forall i, j$, depending on the path taken between θ_{new} and θ_{goal}, a collision cannot be ruled out.

We therefore invoke the routine *ExtendFurther()* in Algorithm 4, which segments the path between θ_{new} and θ_{goal} into smaller segments of appropriate stepsize, and sequentially checks whether the nodes of these smaller segments are valid. If one of the nodes is not valid, the algorithm returns to the main loop and adds the recently accrued valid nodes to the tree. Specifically, if $\|\delta p_{\max}\| \leq \Delta$, the tree is extended toward θ_{goal} in the direction $\hat{v} = (\theta_{\text{goal}} - \theta_{\text{new}})/\|\theta_{\text{goal}} - \theta_{\text{new}}\|$, with stepsize determined from (17), while checking for collisions with obstacles and joint limits (lines 2–4). If a collision occurs or joint limits are exceeded at the extended node θ'_{new}, then the algorithm

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on December 15,2025 at 09:28:35 UTC from IEEE Xplore. Restrictions apply.

TABLE I
RRT STATISTICS FOR A TEN-DOF PLANAR ROBOT

	stepsize	Δ	$\|\delta p_{\max}\|$		# iterations	time(s)	path length
			mean	max			
RRT joint space metric	0.5	–	1.55 ± 0.95	4.37 ± 0.75	6090.9 ± 4654.0	0.74 ± 0.79	122.9 ± 32.1
RRT joint space metric	0.3	–	0.97 ± 0.63	3.32 ± 0.55	95767.4 ± 81743.6	52.51 ± 70.44	309.7 ± 79.3
RRT Cartesian space metric	0.07	1.5	0.30 ± 0.20	1.34 ± 0.14	4254.8 ± 1722.1	0.61 ± 0.39	274.5 ± 60.5
RRT Cartesian space metric	0.06	1.5	0.26 ± 0.18	1.35 ± 0.15	5199.7 ± 1840.8	0.82 ± 0.46	323.6 ± 65.5
RRT adaptive stepsize $\|\cdot\|_{1,2}$	–	1.5	0.30 ± 0.20	1.18 ± 0.20	3337.1 ± 1282.0	0.46 ± 0.27	259.4 ± 60.5
RRT adaptive stepsize $\|\cdot\|_{2,2}$	–	1.5	0.15 ± 0.10	0.67 ± 0.07	7552.9 ± 2621.5	2.72 ± 1.21	495.3 ± 110.4
RRT-Connect joint space metric	0.07	–	0.30 ± 0.19	0.94 ± 0.09	3655.9 ± 1127.7	0.27 ± 0.13	242.1 ± 53.3
RRT-Connect joint space metric	0.075	–	0.33 ± 0.21	1.01 ± 0.10	3310.9 ± 983.8	$\mathbf{0.23 \pm 0.11}$	227.0 ± 45.7
RRT-Connect Cartesian space metric	0.07	1.5	0.32 ± 0.21	1.38 ± 0.10	3689.4 ± 1141.7	0.29 ± 0.14	229.4 ± 48.4
RRT-Connect adaptive stepsize	–	1.5	0.32 ± 0.23	1.37 ± 0.06	$\mathbf{2870.5 \pm 833.3}$	$\mathbf{0.23 \pm 0.10}$	$\mathbf{206.7 \pm 40.7}$

terminates (lines 4 and 12). Otherwise, the algorithm continues to extend the node toward θ_{goal} until the distance $\|\theta'_{\text{new}} - \theta_{\text{goal}}\|$ is less than the stepsize $\|\delta\theta\|_{\theta=\theta'_{\text{new}}}$ evaluated at θ'_{new}, at which point the two nodes are connected.

IV. EXPERIMENTS

Our proposed adaptive stepsize RRT algorithm is evaluated through two sets of numerical experiments, the first involving a ten-dof planar open chain, the second involving a standard seven-axis industrial robot design. For the numerical experiments we compare our adaptive stepsize RRT algorithm with two versions of the standard fixed stepsize RRT algorithm and two versions of the RRT-Connect algorithm (in the latter case, the two versions correspond to using a joint space metric versus a Cartesian space metric for the connectivity test). We give a goal bias [16] of probability 5% toward the goal node. Like most RRT-based panning methods, our algorithm is implemented as a bi-directional search algorithm. A non-incremental 3D collision checking algorithm is used for our numerical experiments. The numerical experiments for the ten-dof planar chain and the seven-axis industrial robot arm are performed using code written in C++ and running on an Intel Core i7-4790 3.50 GHz platform with 24 GB RAM.

Although evaluating the operator-induced norm is in general NP-hard, those induced from the one- and two-norms are straightforward to compute [14], [15]. For our experiments we take the one-norm $\|\cdot\|_1$ in the input configuration space and the two-norm $\|\cdot\|_2$ in the Cartesian task space; the corresponding induced operator norm $\|\cdot\|_{1,2}$ is then evaluated as

$$\|A\|_{1,2} = \max_j \left(\sum_i |a_{ij}|^2 \right)^{1/2}. \tag{21}$$

Pseudo code for calculating $\|A\|_{1,2}$ is provided in Algorithm 5, where θ denotes the current configuration , $\mathcal{B} = \{\mathcal{B}_1, \ldots, \mathcal{B}_m\}$ is the set of link vertices, and $|\mathcal{B}_i|$ is the cardinality of the set $\mathcal{B}_i$. See, e.g., [13] for a treatment of how to convert Denavit-Hartenberg parameter-based forward kinematics into product of exponentials form. For comparison purposes we also perform the same set of experiments using $\|\cdot\|_2$ for the input configuration space. We observe $\|A\|_{2,2}$ is bigger than $\|A\|_{1,2}$, and that

Algorithm 5: OperatorNorm_12($\theta, \mathcal{B}$).

```
a_max ← −inf
J_S ← getSpaceJacobian(θ)
For i = 1 to m
  For j = 1 to |B_i|
    For k = 1 to i
      ret ← ||ω_k × p_ij + v_k||_2
      if ret > a_max then
        a_max ← ret
      end if
    end For
  end For
end For
return a_max
```

all measures, with the exception of elapsed time, vary by a factor of two; the mean and maximum values of $\|\delta p_{\max}\|$ are reduced by half while the number of iterations and path length are doubled. Results of experiments using the two-norm are included in Table I for comparison purposes.

A. Ten-DOF Planar Robot

We consider a planar ten-dof open chain consisting of ten unit-length links connected by revolute joints. Four circular obstacles of radius 2 are placed symmetrically about the robot base as shown in Fig. 3(a). In the initial configuration the arm is outstretched along the horizontal axis (indicated in blue); in the goal configuration (indicated in red), the arm is outstretched along the vertical axis.

For our first set of experiments, we fix the stepsize to 0.5 and perform 100 trials of the standard fixed stepsize RRT algorithm in joint space; results are labelled "RRT joint space metric" in Table I and one result from a single trial run is shown in Fig. 3(b). The mean value of $\|\delta p_{\max}\|$ taken over each step is 1.55 ± 0.95, while the maximum value of $\|\delta p_{\max}\|$ averaged over 100 trials is 4.37 ± 0.75. For the fixed stepsize value 0.5, in the latter case the mean maximum displacement exceeds the diameter of the obstacle, and as can be seen from the results, the planner is unable to detect obstacles, producing infeasible paths that

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on December 15,2025 at 09:28:35 UTC from IEEE Xplore. Restrictions apply.

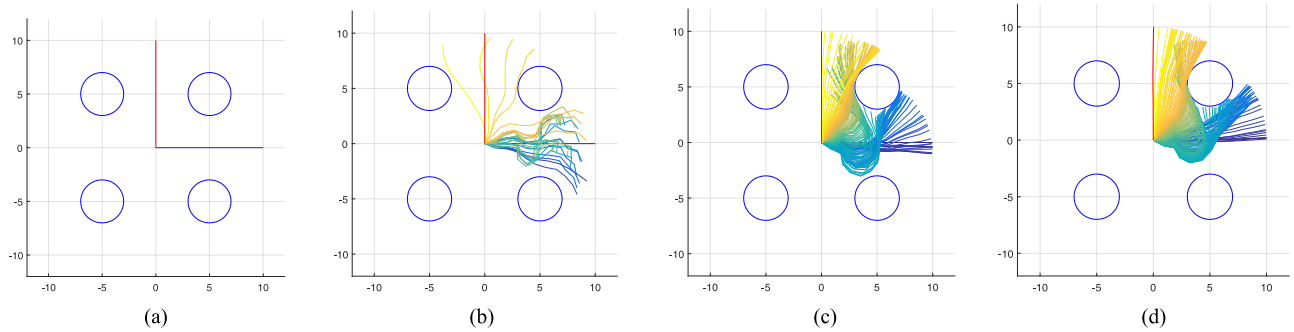


Fig. 3. Experiments with a ten-dof planar open chain. (a) Circular obstacles are placed symmetrically about the robot base, the initial arm posture is shown in blue, and the final posture is shown in red. (b) Results for standard RRT in joint space with stepsize 0.5; observe that collisions occur with the upper-right obstacle. (c) Results for adaptive stepsize RRT in joint space with $\Delta = 1.5$. (d) Results for adaptive stepsize RRT in latent space with $\Delta = 1.5$.

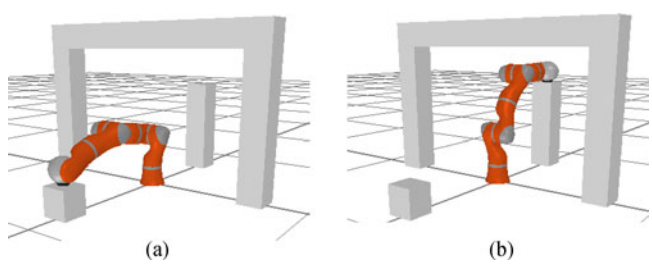


Fig. 4. Experiments with a seven-axis industrial robot arm: (a) Initial configuration; (b) Goal configuration.

result in collisions. Reducing the stepsize further to 0.3, the standard fixed stepsize RRT algorithm still cannot detect the workspace obstacles and fails to produce feasible collision-free paths, despite a significant increase in the number of iterations and computation times as seen from Table I.

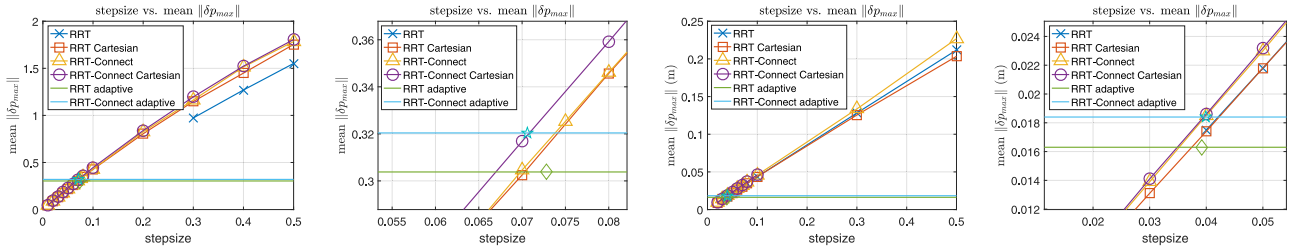
We also pair the standard fixed stepsize joint space RRT algorithm with the Cartesian space metric condition $\|\delta p_{\max}\| \leq \Delta$ for the node connection test: for each link vertex, we calculate its maximum Cartesian space displacement; the nodes are connected if this value is less than the user-specified $\Delta = 1.5$. To find an appropriate fixed stepsize for meaningful comparison, the mean value of $\|\delta p_{\max}\|$ is measured by gradually decreasing the stepsize as shown in Fig 5. In this case the stepsize is selected such that the mean value of $\|\delta p_{\max}\|$ is similar to the corresponding value obtained for our adaptive stepsize RRT when $\Delta = 1.5$. Because our user-specified parameter is Δ, which limits $\|\delta p_{\max}\|$, we compare the cases where the mean values of $\|\delta p_{\max}\|$ are similar. The average stepsize of our RRT adaptive stepsize algorithm is also indicated by diamonds. Results of these trials are also summarized in Table I (labelled as "RRT Cartesian space metric"). Compared to fixed stepsize RRT using a joint space metric for the connection test, the RRT Cartesian space metric algorithm has significantly better convergence. Even with the stepsize reduced to a fourth of its initial value 0.3, the number of iterations is considerably reduced and the elapsed time is decreased by approximately 95%. The results imply that despite the additional computation imposed by the calculation of Cartesian displacements for each link vertex, performance improves considerably. It is worth emphasizing, however, that in both versions of the fixed stepsize RRT algorithm, it is difficult to know in advance an appropriate stepsize; often the stepsize must be chosen in a cumbersome trial-and-error fashion. As shown in the Fig. 6(a), by trial-and-error we find that when the stepsize is less than 0.1, the maximum value of $\|\delta p_{\max}\|$ becomes smaller than 1.5, resulting in a feasible path. Note that for algorithms using Cartesian connectivity conditions, the maximum value of $\|\delta p_{\max}\|$ converges to the Δ value set a priori. This is a natural outcome of not including the ExtendFurther function in Algorithm 4.

We now perform 100 trials of our adaptive stepsize RRT algorithm, setting $\Delta = 1.5$ and using (17) to automatically determine the stepsize. As shown in Table I, the mean value of $\|\delta p_{\max}\|$ for our adaptive stepsize RRT is 0.30 ± 0.20. When the stepsize for the RRT Cartesian space metric algorithm is fixed to 0.07, the mean value of $\|\delta p_{\max}\|$ is similar to that of our adaptive stepsize RRT. Comparing these cases, our adaptive stepsize RRT has the smallest number of iterations and also the shortest planning times. (The additional step (Algorithm 4) needed in the adaptive stepsize RRT algorithm is counted as part of the number of iterations). The path length, which is calculated in joint space, is also the shortest when adaptive stepsize RRT is applied.

Our adaptive stepsize RRT algorithm is also applied and compared to RRT-Connect. Similar to the previous experiments, we set $\Delta = 1.5$; the stepsizes for RRT-Connect are then set to 0.07 and 0.075 (so that the mean values of $\|\delta p_{\max}\|$ are similar to those obtained for our adaptive stepsize RRT algorithm). Results are also shown in Table I. Compared to our original adaptive stepsize RRT method, RRT-Connect has fewer iterations and also shorter planning times. Planning times increase slightly when the Cartesian space metric condition is applied to RRT-Connect, but the number of iterations is similar. Path lengths are also found to decrease.

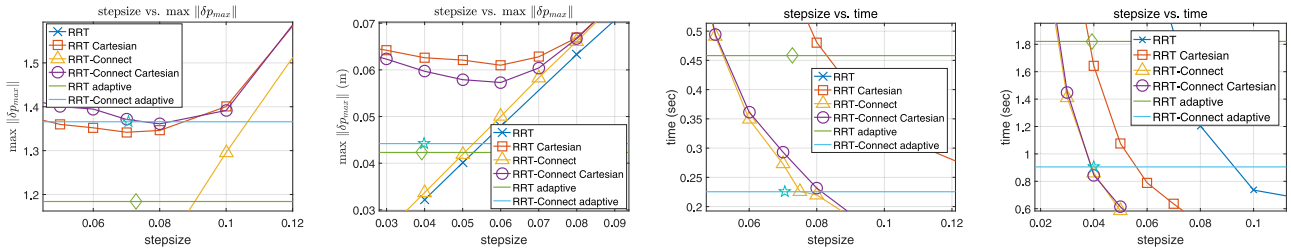
Finally, we combine our adaptive stepsize RRT method with RRT-Connect. The RRT-Connect adaptive stepsize algorithm has the smallest number of iterations, the shortest planning times, and the shortest path lengths. Again, it is worth emphasizing that in standard fixed stepsize RRT algorithms, the stepsize needs to be specified a priori by the user, typically in trial-and-error fashion, while in our adaptive stepsize RRT algorithm, only the very intuitive obstacle minimum size parameter Δ needs to be specified.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on December 15,2025 at 09:28:35 UTC from IEEE Xplore. Restrictions apply.



(a) 10-dof, $\Delta = 1.5$ (b) 10-dof, $\Delta = 1.5$ (c) 7-dof, $\Delta = 0.07m$ (d) 7-dof, $\Delta = 0.07m$

Fig. 5. Stepsize versus mean $\|\delta p_{\max}\|$. Average stepsizes for RRT with adaptive stepsize and RRT-Connect with adaptive stepsize are marked with diamonds and stars, respectively. (a) 10-dof planar robot. (b) An enlarged view of the region of overlap for the 10-dof planar robot. (c) 7-dof industrial arm. (d) An enlarged view of the region of overlap for the 7-dof industrial arm.



(a) 10-dof, $\Delta = 1.5$ (b) 7-dof, $\Delta = 0.07m$ (c) 10-dof, $\Delta = 1.5$ (d) 7-dof, $\Delta = 0.07m$

Fig. 6. Stepsize versus maximum $\|\delta p_{\max}\|$ and time. Average stepsizes for RRT with adaptive stepsize and RRT-Connect with adaptive stepsize are marked with diamonds and stars, respectively. (a) Maximum $\|\delta p_{\max}\|$ for 10-dof planar robot. (b) Maximum $\|\delta p_{\max}\|$ for 7-dof industrial arm. (c) Elapsed time for 10-dof planar robot. (d) Elapsed time for 7-dof industrial arm.

TABLE II
RRT STATISTICS FOR A SEVEN-AXIS ROBOT ARM

	stepsize	Δ	$\|\delta p_{\max}\|$ (cm)		# iterations	time(s)	path length
			mean	max			
RRT joint space metric	0.1	–	4.36 ± 1.56	7.82 ± 0.52	9500.7 ± 8077.6	0.72 ± 1.08	254.7 ± 88.0
RRT Cartesian space metric	0.1	7	4.35 ± 1.61	8.14 ± 0.51	5540.4 ± 2408.8	0.33 ± 0.21	273.9 ± 93.6
RRT Cartesian space metric	0.0375	7	1.63 ± 0.65	6.28 ± 0.66	20978.5 ± 6839.3	1.84 ± 0.86	816.1 ± 255.2
RRT adaptive stepsize	–	7	1.63 ± 0.69	4.20 ± 0.32	19450.5 ± 6277.1	1.74 ± 0.82	814.0 ± 258.6
RRT-Connect joint space metric	0.04	–	1.85 ± 0.86	3.37 ± 0.14	10169.7 ± 3047.0	0.86 ± 0.33	706.4 ± 149.4
RRT-Connect Cartesian space metric	0.04	7	1.86 ± 0.84	5.97 ± 0.68	$\mathbf{10120.6 \pm 3038.7}$	$\mathbf{0.85 \pm 0.33}$	$\mathbf{703.3 \pm 148.7}$
RRT-Connect adaptive stepsize	–	7	1.84 ± 0.96	4.42 ± 0.21	10283.0 ± 3076.1	0.91 ± 0.35	707.1 ± 146.4

B. Seven-DOF Industrial Robot Arm

We now evaluate the performance of our adaptive stepsize algorithm for a seven-axis industrial robot manipulator of the type shown in Fig. 4. For easier calculation of the positions of the link vertices, each link is modeled as a three-dimensional rectangular box. The objective is to find a collision-free path from the initial (left figure) to the final configuration (right figure). The robot's maximum length is set to 1 m, and for this problem we set $\Delta = 7$ cm for the variable stepsize algorithm. For the standard RRT algorithm, we set the fixed stepsize initially to 0.1 and successively reduce its value to 0.02 (memory overflow occurs due to the high number of nodes generated, thereby preventing any further reduction of the stepsize). The results of our numerical experiments are summarized in Table II.

For the standard fixed stepsize RRT algorithm, we first compare the use of the joint space metric versus the Cartesian space metric for the stepsize value 0.1. Using the Cartesian space metric leads to a nearly two-fold improvement in efficiency. However, note that for this case the maximum value of $\|\delta p_{\max}\|$ (8.14 ± 0.51) is greater than Δ (7.0). What this implies is that collisions with small objects may potentially occur that could go undetected. This example highlights one of the pitfalls of using the Cartesian space metric without using the accompanying inequality bound: the range of $\|\delta p_{\max}\|$ cannot be accurately estimated since the bound is not known. Furthermore, when the stepsize is 0.3 and 0.5, it fails to satisfy the Cartesian condition and oscillates near the goal, failing to connect the two trees. After repeated numerical trials, we determine that a stepsize value of 0.0375 leads to a mean value $\|\delta p_{\max}\|$ of 1.63 ± 0.65, which is approximately equal to the mean $\|\delta p_{\max}\|$ when Δ of adaptive stepsize RRT is 7. The mean values of $\|\delta p_{\max}\|$ for the changes in stepsize are shown in Fig. 5. As shown in Fig. 6(b), when

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on December 15,2025 at 09:28:35 UTC from IEEE Xplore. Restrictions apply.

the stepsize becomes smaller than 0.085, the maximum value of $\|\delta p_{\max}\|$ becomes smaller than $\Delta = 7$ cm, and the resulting path becomes feasible.

The computational performance of our adaptive stepsize RRT algorithm is approximately of the same order as the fixed stepsize RRT algorithm with stepsize values in the range 0.035–0.04. Compared to when the fixed stepsize is set to 0.0375, the number of iterations for the adaptive stepsize RRT algorithm is reduced by about 7%, while the planning time is reduced by about 5%. Note, however, that the maximum displacement for the adaptive stepsize case (4.20 ± 0.32) is smaller than the the corresponding maximum displacements for the fixed stepsize case (6.28 ± 0.66).

A performance comparison of RRT-Connect versus our adaptive stepsize algorithm is shown in Table II. With $\Delta = 7$, all of the performance indices, i.e., the number of iterations, planning time, and path length, between the adaptive stepsize RRT-Connect algorithm and RRT-Connect are very similar. The results suggest that the advantages of the adaptive stepsize RRT algorithm appear to amplify with increasing degrees of freedom.

V. Conclusion

Using a standard operator norm inequality and exploiting the structure of an open chain's forward kinematics equations as captured by the product of exponentials formula [9], we have presented an adaptive stepsize RRT planning algorithm for open kinematic chains. The key result is an approximate bound on the Cartesian displacement of the open chain tip for a given joint space displacement. This bound, which can be computed in real-time and is useful in other motion planning contexts described below, is used to compute, for a given robot configuration, approximate bounds on the maximal deviations of the points on each of the robot's links (each link is modeled as a convex polytope). Not having to manually set the stepsize parameter through a time-consuming trial-and-error process is one of the important advantages of our algorithm.

One by-product of our approach is a Cartesian space metric that can be used to determine whether or not to connect two nodes in standard fixed-stepsize RRT planning algorithms. Despite the increased computation, in many cases the overall performance of the algorithm is enhanced compared to the standard joint space metric threshold test that is more commonly used. The main drawback of having to manually determine an appropriate stepsize still remains, however.

Our adaptive stepsize algorithm has been validated through extensive numerical experiments with a ten-dof planar open chain and a seven-axis industrial arm manipulator. The computation times are generally faster than fixed-stepsize RRT algorithms, and the advantages tend to become more pronounced for robots with higher degrees of freedom. The fact that the stepsizes can be automatically determined simply by the user specifying the size Δ of the smallest workspace obstacles is the most obvious advantage of our adaptive stepsize algorithm. Our experimental studies further indicate that in many cases the actual maximum displacements are less than the specified Δ, confirming the conservative nature of the displacement bound used.

Recently, for robots with many degrees of freedom, machine learning-based dimension reduction methods, in which a lower-dimensional representation of the configuration space (the latent space) is constructed from a collection of representative motions, have become a popular means of reducing the complexity of the planning problem. Provided the mapping from the latent space to the configuration space is well-characterized and smooth, our method can also be extended to this situation as well.

The bounds used in our algorithm can also be used in other motion planning contexts. Regions in the latent space corresponding to collisions may need to be mapped out, usually by sampling methods; our bound can be used to determine an appropriate sampling resolution from knowledge of the sizes of the obstacles relative to the robot. Although our results have only been established for open chains, robots with more complex structures like closed loops and nonconvex link shapes can likely be modeled with appropriate extensions.

Acknowledgment

Detailed comments and suggestions from the reviewers are also greatly appreciated.

References

[1] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Comput. Sci. Dept., Iowa State Univ., Tech. Rep. TR 98–11, 1998.

[2] S. M. LaValle, "Motion planning: The essentials," *IEEE Robot. Autom. Mag.*, vol. 18, no. 1, pp. 79–89, Mar. 2011.

[3] S. M. LaValle, "Motion planning: Wild frontiers," *IEEE Robot. Autom. Mag.*, vol. 18, no. 2, pp. 108–118, Jun. 2011.

[4] M. Elbanhawi and M. Simic, "Sampling-based robot motion planning: A review," *IEEE Access*, vol. 2, pp. 56–77, 2014.

[5] J. J. Kuffner and S. M. LaValle, "RRT-Connect: An efficient approach to single-query path planning," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2000, pp. 995–1001.

[6] I. A. Şucan and L. E. Kavraki, "On the implementation of single-query sampling-based motion planners," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2010, pp. 2005–2011.

[7] C. Wang and M. Q.-H. Meng, "Variant step size RRT: An efficient path planner for UAV in complex environments," in *Proc. IEEE Int. Conf. Real-time Comput. Robot.*, 2016, pp. 555–560.

[8] M. J. McCourt, C. T. Ton, S. S. Mchta, and J. W. Curtis, "Adaptive stepsize RRT algorithm for improved coverage," in *Proc. AIAA Guid., Navig., Control*, 2016, p. 0638, https://arc.aiaa.org/doi/abs/10.2514/6.2016-0638.

[9] R. W. Brockett, "Robotic manipulators and the product of exponentials formula," in *Proc. Int. Symp. Math. Theory Netw. Syst.*, Berlin, Germany: Springer, 1984, pp. 120–129.

[10] J. Wang, A. Hertzmann, and D. M. Blei, "Gaussian process dynamical models," in *Proc. Neural Inf. Process. Syst.*, 2005, pp. 1441–1448.

[11] J. B. Conway, *A Course in Functional Analysis*, vol. 96, Berlin, Germany: Springer, 1990.

[12] R. M. Murray, Z. Li, and S. S. Sastry, *A Mathematical Introduction to Robotic Manipulation*. Boca Raton, FL, USA: CRC Press, 1994.

[13] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge, U.K.: Cambridge Univ. Press, 2017.

[14] J. Rohn, "Computing the norm $\|A\|_{\infty,1}$ is NP-hard??" *Linear Multilinear Algebr.*, vol. 47, no. 3, pp. 195–204, 2000.

[15] J. A. Tropp, "Topics in sparse approximation," Ph.D. dissertation, Univ. Texas, Austin, TX, USA, 2004.

[16] S. M. LaValle and J. J. Kuffner, "Rapidly-exploring random trees: Progress and prospects," in *Algorithmic and Computational Robotics: New Directions 2000 WAFR*. Natick, MA, USA: AK Peters, 2001, pp. 293–308.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on December 15,2025 at 09:28:35 UTC from IEEE Xplore. Restrictions apply.